SOFE 3720 - Introduction to AI

Final Project

-

Pathfinding Visualizer Using A*, Greedy Best-First Search, and Uniform Cost Search

Friday, March 21, 2026

Final Project Group - CRN 74674 - 3

Member Names	Student Number
Abdul Aziz Syed	100792709
Areeb Khan	
Nehal Rauf	

Introduction

This project idea is to make a Pathfinding Visualizer on a 2D grid. The content is to let a user place a start node, an end node, and walls, then see how different AI search algorithms find a path from start to end.

The system compares *A*, Greedy Best-First Search,* and *Uniform Cost Search (UCS)* on the same grid so their actions can be compared together; which fulfills the project requirements of a grid-world pathfinder and comparing different search methods.

The problem being solved is a *shortest-path problem*. The agent needs to move from 1 point to another while avoiding blocked cells. Even on a simple grid, there can be many possible routes, even more so when walls are added.

The system needs to search through possible moves and decide on which path is best based on search rules, path cost, and/or a heuristic.

Objectives

The goal of this project is to design and build an interactive AI system that can solve and visualize a pathfinding problem on a grid and understand how different search methods act and which are more efficient than others in certain situations.

<u>The main objectives of the project are:</u>

- ➢ to model pathfinding as a search problem,
- ➢ to implement A*, Greedy Best-First Search, and UCS algorithms in our code,
- ➢ to give the user a visual interface to place the start node, end node, and walls,
- ➢ to animate how each algorithm explores the grid,
- ➢ to compare the algorithms using path length, explored nodes, and runtime,
- ➢ to organize the code using a clean and modular design.

Program Development/Problem Solution

State Space

The state space is made up of all valid cells in the grid that can be visited. Each cell is treated as a node. In the code, every node stores its position and other values needed for search, such as its cost values, parent, and whether it is a wall, the start, or the goal.

Initial State

The initial state is the start node chosen by the user. This is stored in the grid as *start_node*.

Goal State

The goal state is the end node chosen by the user. This is stored in the grid as *end_node*.

Actions

From any valid node, the agent can move: up, down, left, or right. Diagonal movement isn't allowed. This is controlled by the *get_neighbors()* function in the code.

Constraints

The search has a few clear limits:

- the agent can't leave the grid
- the agent can't move into wall cells
- the start and end nodes can't be walls
- and only 1 start node and 1 end node can exist at a time

Cost Function

Each move has a cost of 1, so the total path cost is the number of steps taken to reach the goal. In the code, this is done by increasing the path cost by 1 whenever a neighbour is explored.

Why AI Is Needed for the Problem

AI is for the system to make smart decisions about which nodes to explore instead of checking routes randomly. Once the grid gets complex, like with many walls, the number of possible paths grows quickly. Search algorithms can help solve this problem in a more organized and efficient way. A* uses both cost and heuristic information, Greedy mainly follows the heuristic, and UCS depends only on path cost.

System Design

The project guidelines say the design section should include at least a *use case diagram* and a *class diagram,* and your uploaded diagrams already cover that. They also ask for architecture and data flow, which can be explained in words in the report.

Use Case Diagram

The use case diagram shows 1 main actor, the user, interacting with the Pathfinding Visualizer. The user can:

- set the start node,
- set the end node,
- place or remove walls,
- run the algorithms,
- compare results,
- and reset the grid.

These actions are all supported in the actual program. The interface includes controls for drawing walls, selecting the start and end nodes, clearing paths, resetting the grid, and running all algorithms for comparison. So the use case diagram is not just theoretical. It matches the real features in the program.

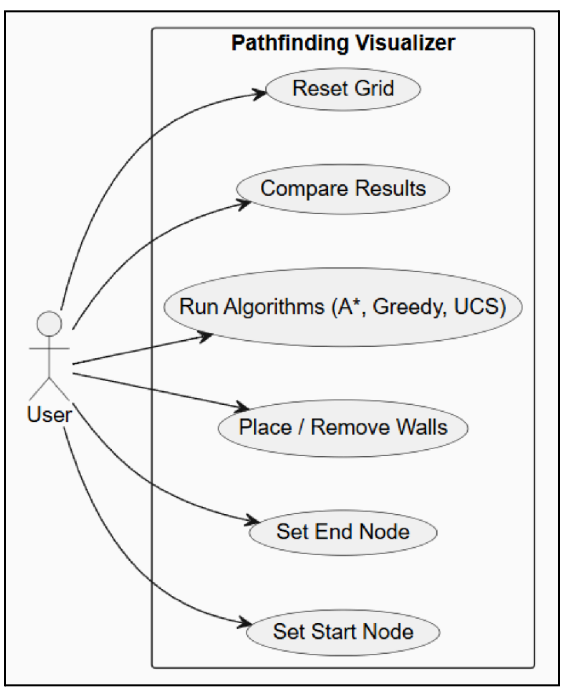

Figure 1. Use case diagram showing how the user interacts with the pathfinding visualizer.

Class Diagram

The class diagram includes the main classes used in the program where the search logic is kept separate from the user interface:

- VisualizerApp: handles the user interface, animation, and result display
- Grid: manages the full map, including walls, start, and end nodes
- Node: 1 cell in the grid
- PathfindingAlgorithm: the base class for the search algorithms
- AStarAlgorithm: search logic algorithm
- GreedyAlgorithm: search logic algorithm
- UCSAlgorithm: search logic algorithm

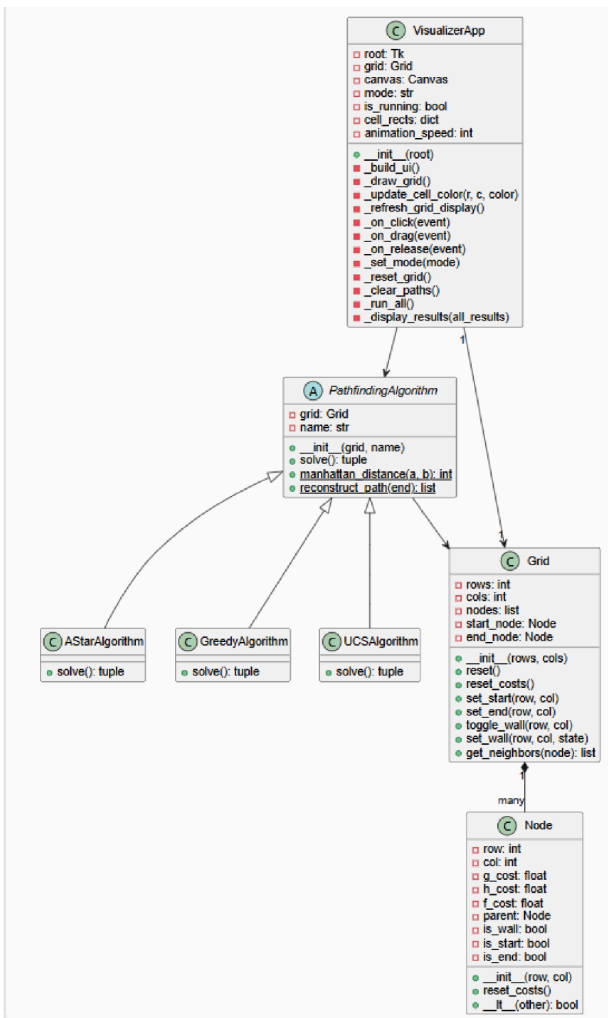

Figure 2. Class diagram showing the main classes and how they relate to each other.

System Architecture

The system works in a flow of steps/stages. First, the user interacts with the grid using the interface. Then the interface updates the internal grid model. When the user clicks “Run All,” the algorithms read the grid, perform the search, and return the explored nodes, final path, and performance data. The interface then animates the results and displays the comparison table.

User Input → VisualizerApp → Grid/Node Model → Search Algorithm → Results → Animation and Comparison Table

Data Flow

The data flow starts when the user clicks on the grid. Those clicks trigger event-handling methods in the program, which update the grid state. After that, the selected algorithms process the grid and search for a path. Once they finish, the results are passed back to the visualizer, which colors the explored cells, shows the final path, and updates the performance area on the right side of the window.

Methods

In this project, the algorithms were implemented directly in Python. Standard Python tools such as *heapq*, *time*, and *tkinter* are used only to support the program, not replace the real search logic.

All 3 algorithms use the same grid and node structure and share some common ideas:

- each node keeps track of cost values
- neighbours are found through the grid
- parent links are used to rebuild the final path
- a priority queue is used to decide which node to explore next
- and the code records the explored order, final path, and performance values

A* Search

A* uses both the real path cost and a heuristic estimate to guide the search. In the code:

- *g_cost* is the cost from the start node to the current node,
- *h_cost* is the estimated remaining distance to the goal,
- *f_cost = g_cost + h_cost*

The heuristic used is *Manhattan distance*, because the agent can only move up, down, left, or right. A* is the best choice in this kind of grid because it balances efficiency and path quality. It doesn't randomly search for the goal or ignore useful direction information.

Greedy Best-First Search

Greedy Best-First Search uses only the heuristic value. In this implementation, it sets:

- *f_cost = h_cost*

That means it mainly focuses on whichever node looks closest to the goal. This can make it seem fast, especially in open areas, but it can also make poor choices when walls force a longer route. It is useful for the Pathfinding Visualizer because it shows what happens when a search method depends too much on the goal estimate and not enough on the real path cost.

Uniform Cost Search

Uniform Cost Search uses only the actual path cost:

- *f_cost = g_cost*

This means it expands the cheapest path found so far without using any guess about how close the goal is. UCS is reliable and cost-based, but without heuristics, it checks more nodes than A*. It is still important in this project because it gives a solid baseline for comparison.

Implementation

The program is written in Python and uses *Tkinter* for the graphical interface. The grid is set to 25 rows and 25 columns, with each cell drawn visually on the canvas. The interface includes buttons for drawing walls, setting the start node, setting the end node, clearing paths, resetting the grid, and running all algorithms. It also includes a speed slider and a performance comparison area.

The VisualizerApp class controls the interface and user actions. The Grid class stores all nodes and handles operations like setting the start node, setting the end node, creating walls, and getting neighbours. The Node class stores the state of each cell. The three algorithm classes contain the actual search logic.

One strong part of the implementation is that the program does not just run one algorithm. It runs all three one after another, animates them, then shows the results together. This makes the comparison much clearer and makes the project more useful during the demo.

The program also handles missing setup properly. If the user tries to run the search before setting both the start and end nodes, a warning message is shown. That improves the user experience and keeps the program from running in an invalid state.

Results

This section should include your actual test results from the program. The easiest way is to run several different grid layouts and record:

- nodes expanded
- path length
- time in milliseconds

You can test a few cases like:

- empty grid
- moderate obstacles
- maze-like walls
- heavy obstacle layout
- no-path case

Test Cases

Test Case 1: Empty Grid

This test has no walls. It shows the basic behaviour of the algorithms when the path is open and simple.

Test Case 2: Moderate Obstacles

Some walls are placed between the start and goal so the algorithms need to find a detour.

Test Case 3: Maze-Like Layout

The walls create tighter paths and dead ends. This is useful for showing how the algorithms react when the direct-looking route is not actually the best one.

Test Case 4: High Obstacle Density

This case includes more walls but still leaves a path open. It tests how well the algorithms perform in a more difficult layout.

Test Case 5: No Path

The goal is blocked off so no valid path exists. This checks whether the program can handle failure correctly.

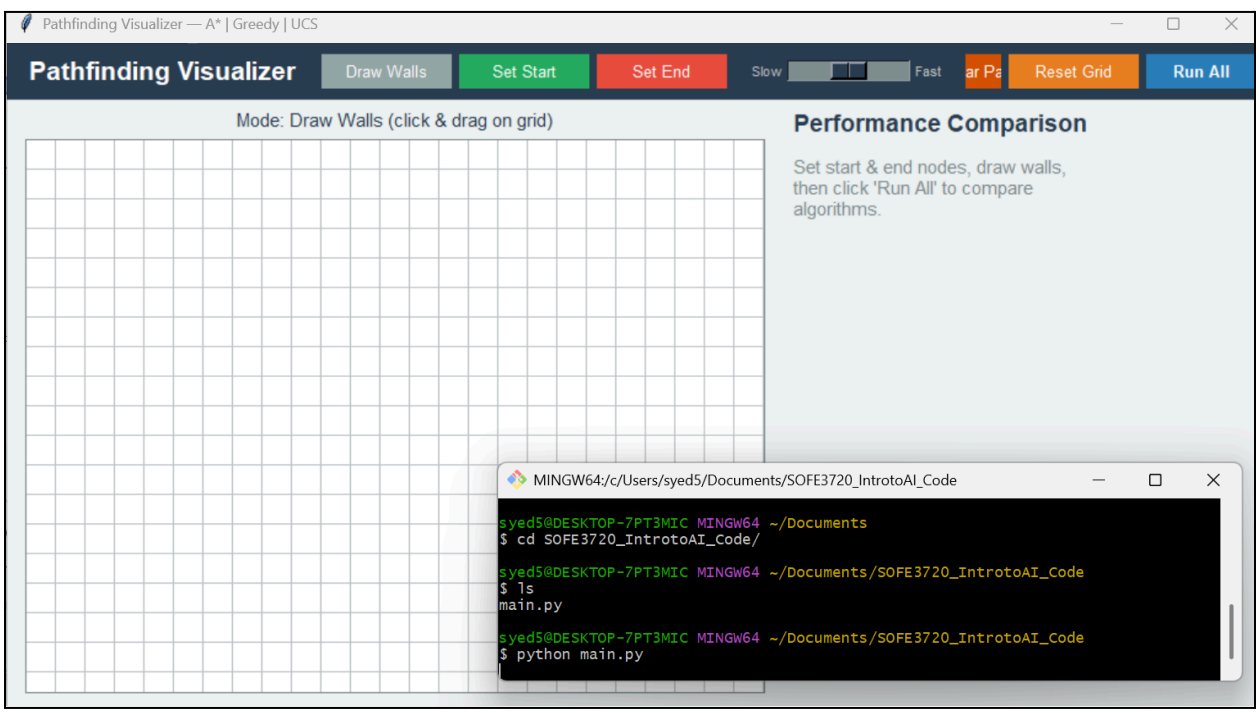

Figure 3. Empty grid test case before running the algorithms.

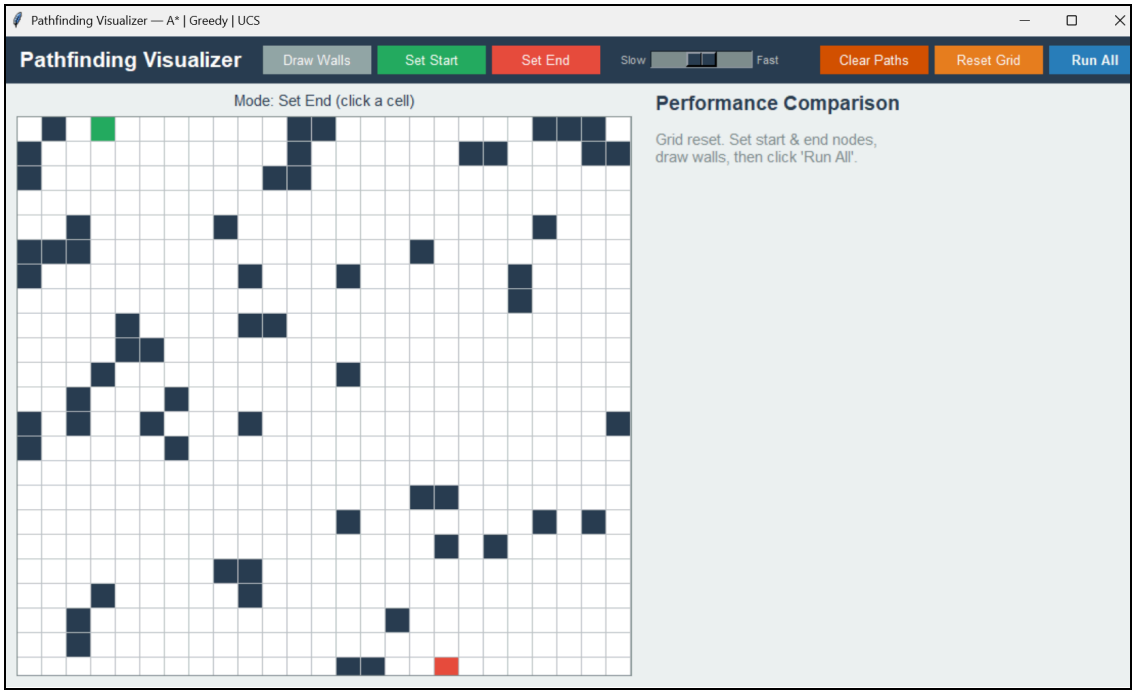

Figure 4. Example test case with walls placed between the start and goal.

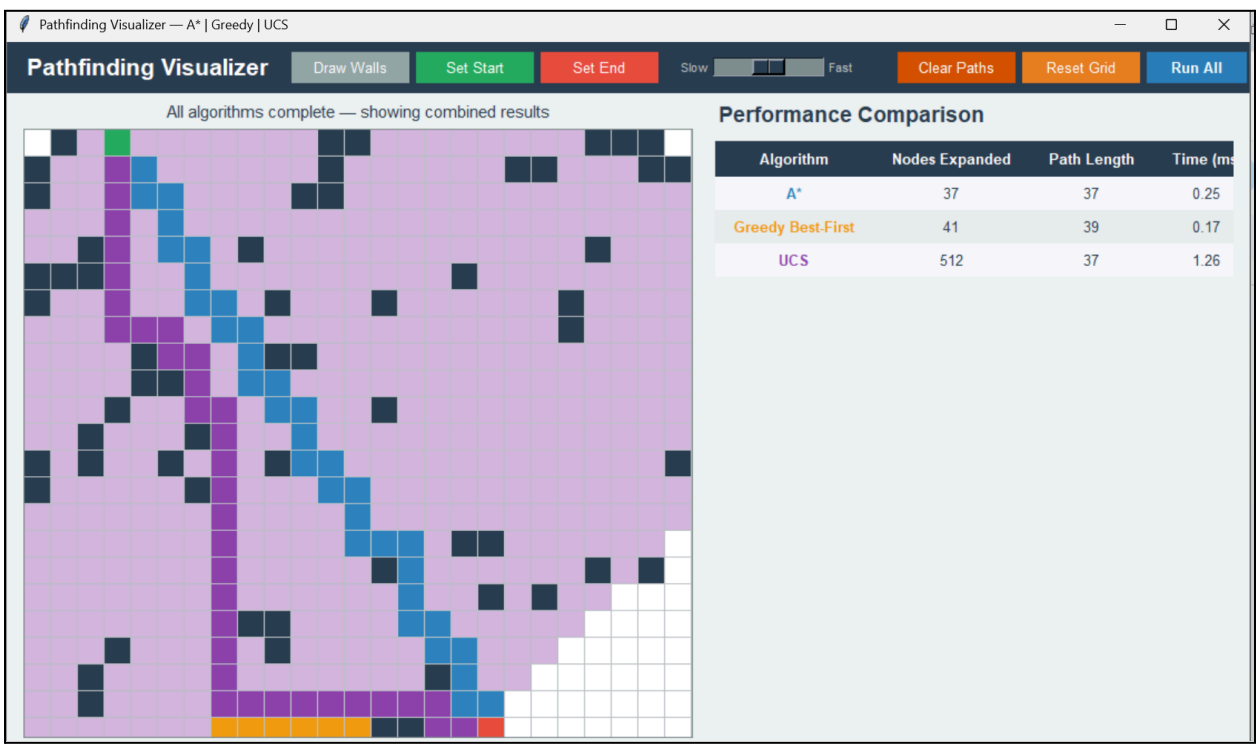

Figure 5. Program output showing explored nodes, final paths, and the performance table.

Result Discussion

The results show that the 3 algorithms act differently even when solving the same problem. They can reach their respective goals in simpler paths, but differences occur when obstacles are added.

- A* usually gives the best balance because it uses both the real path cost and the heuristic, so it tends to stay efficient while still finding a good route.
- Greedy Best-First Search often looks fast because it moves toward the goal aggressively, but it can get misled when the walls force a longer detour. That means it may not behave as well in harder layouts.
- UCS is more careful because it only follows actual path cost, but because it has no heuristic guidance, it often explores more cells before reaching the end.

In the no-path scenario, the program correctly reports that no path exists, meaning that the code implementation can handle failure cases properly as well.

Conclusion

This project successfully built a pathfinding visualizer that compares A*, Greedy Best-First Search, and Uniform Cost Search on a grid-based environment. The system lets the user create different layouts, run the algorithms, and see how they explore the grid and build paths. The final program shows both the visual side of AI search and the measurable side through nodes expanded, path length, and runtime.

Limitations and Future Improvements

Even though the project works well, there are still a few limits. The system only allows 4-directional movement, and diagonal movement isn't supported. All the moves have the same cost, which keeps the problem simple but doesn't represent weighted terrain. The program uses only 3 algorithms, and the Visualizer runs locally as a Tkinter desktop application.

<u>Some future improvements could be:</u>

- adding diagonal movement
- adding weighted cells
- letting the user run the algorithms 1 at a time